

AI 工具使用说明

一、使用的工具

环节	工具	用途
主力开发	Claude Code（命令行 Agent）	生成模拟数据脚本、看板页面、文档初稿；在无头 Chromium 里自查渲染结果
渲染自查	Playwright + Chromium（由 Claude Code 驱动）	打开页面截图、抓 console 报错、抽取页面上的 KPI 文本做断言
交叉校验	Python（自己写的独立脚本）	绕开前端逻辑，独立算一遍所有指标，和页面显示值逐项比对
对抗性评审	另一个大模型（与写代码的不是同一个）	把题目 + 全部源码 + 渲染状态打包成盲评材料交给它，让它以面试官视角挑错
图表库	ECharts 6.1.0（本地 vendor，无 CDN 依赖）	全部图表

没有使用任何低代码/拖拽式 BI 工具，看板是手写的单页应用，所有聚合逻辑可读可查。

二、关键提示词

按实际使用顺序，保留了主干：

- 需求拆解：把题目原文交给 Claude Code，要求先不写码，列出交付物清单、技术选型的取舍（单文件 HTML vs 前端工程 vs Streamlit），并明确指出评审方要"双击能打开"，让它据此推荐方案。
- 数据建模：
 - > 生成 60 天 × 8 个点位的模拟数据，固定随机种子保证可复现。刻意埋几条能被发现的运营故事：某设备整机离线两天、某点位周末排队导致生成时长翻倍、某点位审核拒绝率长期偏高、某设备最近几天模型报错率上升、一次线下活动带来参与人数峰值、以及一台设备此刻仍然离线。
 - 指标口径：
 - > 平均生成时长要能跨天跨点位正确聚合；P95 不能用"每天的 P95 再平均"。给出一个在任意筛选切片下都算得准的存储方案。
 - 看板实现：
 - > 一行筛选控件作用于整页；6 个 KPI 磁贴带环比和迷你趋势线；图表覆盖趋势、点位对比、转化漏斗、审核构成、高峰时段；设备状态面板；异常事件表可按级别/类型/状态筛选并展开处理建议。不允许双轴图；标称类别不做明暗渐变；每张图配一个表格视图。
 - 自查：
 - > 用无头浏览器打开页面，报告 console 错误，抓取 KPI 文本，再用 Python 独立算一遍同样口径的指标，逐项比对，不一致就指出来。

三、本人完成的主要修改

AI 的初稿能跑，但跑得通不等于对。下面是我自己发现并改掉的问题，按性质分三类。

1. 指标口径错误（影响数字正确性）

- 平均生成时长把失败请求也算了进去。失败多半是超时，把它们计入均值会把数字系统性拉高。我改成只统计成功生成，并在指标说明里写明这条口径。
- P95 原本按天存一个值，聚合时再求平均——分位数不能这么算。我改成每条记录存一个 13 档的生成时长直方图，跨切片时先合并直方图再线性插值。注意插值出来的是估算值不是精确分位数——这一点我一开始写成了"都是准的"，后面被评审指出来才改。
- 转化率的分子分母混了口径。生成成功率是"次数"口径（一人可以生成多次），扫码/分享转化率是"人数"口径。我在数据里拆成两组字段，并在文档里逐条标注。

2. 运营视角的缺陷（数字对，但没用）

- 告警按天刷屏。初版对每个越界的日子都生成一条事件，一个持续 60 天的问题产生了 60 条记录，异常列表完全没法看。我改成按连续区间去重：一次持续异常只开一张单，详情里写持续天数——这也是真实告警系统的做法。
- 30 天平均把正在发生的事故抹平了。有个点位最近 5 天成功率跌到 84%，但 30 天聚合后是 94%，看板一条预警都不报。我给规则引擎加了第二层判定：区间合计和范围内最后一天各判一次，只要最后一天越界就标记"仍在发生"并排到最前。这个改动让实时离线的设备和持续衰退的设备都浮了上来。
- 预警的处理建议给错了。"扫码转化率偏低"复用了"素材下载失败"的处置文案，让人去查 CDN——但转化率低是现场引导问题，不是设备故障。我给这条规则单独写了建议动作。

3. 图表渲染缺陷（AI 自己看不见的）

这几个问题只有真的把页面打开看一眼才会发现，AI 写完代码是不会自己去看的：

- 百分比坐标轴显示成了 0.97、0.96。原因是配置合并顺序写反了，公共轴配置覆盖掉了自定义的格式化函数。
- 阈值虚线根本不在可视范围内。坐标轴自动缩放到数据范围（91%–97%），而阈值是 90%，线被画到了画布外面——卡片副标题还写着"虚线以下为低于阈值的日子"。我改成让轴的上下端点强制包含阈值。
- 坐标轴端点印出了 `12.471364357323855s`。加留白后没取整。改成向外取整到干净的刻度。
- 两条趋势线几乎完全重合，末端标签还互相压字。参与人数和生成请求次数本来就高度相关，同时画出来信息量接近零。我换成"参与人数 vs 扫码带走人数"——两条线量级拉得开，而且它们之间的距离直接就是流失掉的人，这才是运营想看的。
- 末端标签被卡片右边界切掉。去掉了逐条末端标签，改成只标一个峰值，身份识别交给图例。

4. 其他工程决策

- `file://` 下 `fetch()` 会被 CORS 拦，评审方双击打开会白屏。数据生成脚本同时产出一份 `mock.js`（挂到 `window.MOCK_DATA`）用 `<script>` 引入，`mock.json` 作为交付物本身保留。ECharts 也放本地 `vendor/`，全程不依赖外网。
- 配色不靠眼睛判断。分类色和漏斗的序数色都跑过对比度与色盲可分辨性校验；状态色（通过/拒绝/在线/离线）一律配文字标签，不让颜色单独承担含义；每张图都有表格视图，数值不依赖悬浮提示。
- 阈值集中在数据的 `meta.thresholds` 里，看板和指标说明文档引用同一份，避免两边写着不同的数。

三点五、第三轮：让另一个模型做对抗性评审

自查只能发现"我想得到要查"的问题。所以做完之后，我把这份作业打包交给另一个大模型做盲评。

做法：把题目原文、全部源码逐字内联、以及用无头浏览器 dump 出来的真实渲染状态（三种筛选下的每个 KPI 数值、七张图的系列与阈值配置、设备表、异常表）做成一份自包含的材料，让它以资深面试官的身份评，并要求每条意见标注严重程度、区分"事实错误"与"主观偏好"、凡是无法运行页面验证的结论必须声明是推测。开发过程的上下文单独放一个文件，让它先盲评、后对照，避免被我的叙述带跑。

它提了六条事实性问题。我逐条复现，全部属实，没有一条误报：

它指出的	我复现的结果	改法
文档称参与人数为"去重人数"，但数据只有天×点位汇总，跨天求和必然重复计人	属实，且是最致命的一条。30 天的 88,576 只能叫人次	给每条记录加了 HyperLogLog 草图：人数和分位数一样不可加，所以存可合并的结构而不是标量。现在去重人数在任意切片下都算得出来，顺带得到人均参与次数
持续型异常只在开始日建事件，事件表按开始日过滤	属实。一条 7/18 起持续 60 天的异常，在默认 30 天视图里预警在报、事件表却查不到	事件带起止日期，改为按区间相交过滤
P95 是估算却被写成"都是准的"；"14 档"实为 13 档	属实。默认切片估算 15.7s vs 真实 15.39s；480 个单日切片最大偏差 +1.27s，且偏差恒为正	UI 与文档一律标注"估算"，并写明偏差恒为正的原因（桶内线性插值假设均匀，真实时长右偏）
"仍在发生"只代表所选范围末日越界；设备状态读数据集末日快照，却与所选日的当日指标并列	属实。筛到 9/5 时设备表显示某台"离线"（它 9/15 才断网），同行"当日参与"却是 9/5 的数	拆成「截至最新数据未恢复」与「范围末日越界」两个标记；设备表最新日用心跳、历史日按当天数据推导且心跳列置空
漏斗从"参与"直接跳到"生成成功"，已有的 <code>gen_users</code> 没用上	属实。合并的那一步里 24.4% 是压根没发起生成（引导问题）、6.3% 是发起了没成功（技术问题）	漏斗补成五段，两类流失分开
自称"看板与文档引用同一份阈值"，实际文档里是手写字符串	属实，是过度声明	改成 指标说明.template.md + <code>build_docs.py</code> ，阈值从数据的 meta 注入，这句话才成立

修复过程中自己又发现两个问题，都是因为写了校验才暴露的：

- HLL 用裸 FNV-1a 哈希时，对 `u00000001` 这类连续 ID 混合不足，n=300 时估算偏低 49%，寄存器下标卡方 2473（理想 1024）。加 `murmur3` 的 `fmix32` 收尾后最大误差降到 3.9%、卡方 1093。这种错不会报异常，只会安静地给出错的数字。
- 单日切片上 HLL 估出的去重人数大于人次，人均算成 0.98 次。用两个已知的硬约束把估算夹住（上界 = 人次，下界 = 单格去重最大值），并在只有一条记录时直接取精确值。

校验脚本也按它的建议进了仓库（它的原话是这比再加一张图更有说服力）：`scripts/verify.sh` 一条命令跑完数据可复现性、指标独立重算、渲染端对账共 60 项断言，其中包含它指出的每一个复现用例。

四、AI 帮了什么，没帮上什么

帮上的：把结构性的活儿压缩得很厉害——数据生成脚本、页面骨架、图表配置、文档初稿，这些如果手写要花掉大半天。让它顺手写一个 Playwright 自查脚本也很划算。

没帮上的：所有需要业务判断的地方它都会给一个"看起来合理"的答案，但未必对——平均时长算不算失败请求、告警要不要抑制、30 天平均会不会掩盖问题，这些只有真的想过运营每天怎么用这块看板才知道。渲染层面的问题它更是完全看不见，必须自己打开页面逐张图看。

更值得说的是它的盲区：写代码的那个模型自查了一轮，没发现"去重人数"这个口径根本立不住——因为它只会检查自己写的逻辑对不对，不会回头质疑口径本身的前提。这个问题是换一个模型、换一个视角（面试官而不是实现者）才暴露出来的。

我的用法是三段：让 AI 写第一版 → 换一个模型以对抗视角挑错 → 我逐条复现后再决定改不改。中间那一步不能省，最后一步也不能省：评审意见同样需要验证，只是这次六条全中而已。真正不可替代的是最后的判断——决定哪些该修、修到什么程度、以及像"去重人数"这种问题是

改称谓还是改数据模型。我选了后者，因为面试官一定会追问"那怎么才能拿到真去重"，而这个答案应该写在代码里，不是靠嘴说。